

TOPIC 2 : BRUTE FORCE

1. Write a program to perform the following

An empty list

A list with one element

A list with all identical elements

A list with negative numbers

Test Cases:

1. Input: []

- Expected Output: []

1. Input: [1]

- Expected Output: [1]

2. Input: [7, 7, 7, 7]

- Expected Output: [7, 7, 7, 7]

3. Input: [-5, -1, -3, -2, -4]

- Expected Output: [-5, -4, -3, -2, -1]

Aim

To write a Python program to sort a list of numbers in **ascending order** and correctly handle special cases such as an empty list, a single-element list, identical elements, and negative numbers.

Algorithm

1. Start.
2. Read the input list.
3. If the list is empty, return the empty list.
4. Otherwise, sort the list in ascending order.
5. Display the sorted list.
6. Stop.

code:

```
# Function to sort the list using Bubble Sort
def bubbleSort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr

# User input
n = int(input("Enter the number of elements: "))

if n == 0:
    arr = []
else:
    print("Enter the array elements:")
    arr = list(map(int, input().split()))

# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    sortedArray = bubbleSort(arr)
    print("Sorted List:", sortedArray)
```

input and output:

```
Enter the number of elements: 4
Enter the array elements:
2 6 -5 4
Sorted List: [-5, 2, 4, 6]
```

- Time Complexity : $O(n \log n)$
- Space Complexity : $O(n)$

Result

The program successfully sorts the given list in ascending order while correctly handling empty lists, single-element lists, identical elements, and negative numbers.

2. Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.

Sorting a Random Array:

Input: [5, 2, 9, 1, 5, 6]

Output: [1, 2, 5, 5, 6, 9]

Sorting a Reverse Sorted Array:

Input: [10, 8, 6, 4, 2]

Output: [2, 4, 6, 8, 10]

Sorting an Already Sorted Array:

Input: [1, 2, 3, 4, 5]

Output: [1, 2, 3, 4, 5]

Aim

To write a Python program to sort an array in **ascending order** using the **Selection Sort** algorithm.

Algorithm

1. Start.
2. Read the array of elements.
3. Divide the array into a sorted and an unsorted region.
4. For each position from the beginning of the array:
 - Find the smallest element in the unsorted region.
 - Swap it with the first element of the unsorted region.
5. Repeat until all elements are sorted.
6. Display the sorted array.
7. Stop.

code:

```
# Function to perform Selection Sort
def selectionSort(arr):
    n = len(arr)

    for i in range(n):
        minIndex = i

        # Find the index of the smallest element
        for j in range(i + 1, n):
            if arr[j] < arr[minIndex]:
                minIndex = j

        # Swap the smallest element with the current element
        arr[i], arr[minIndex] = arr[minIndex], arr[i]

    return arr

# User input
n = int(input("Enter the number of elements: "))

if n == 0:
    arr = []
else:
    print("Enter the array elements:")
    arr = list(map(int, input().split()))

# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    sortedArray = selectionSort(arr)
    print("Sorted Array:", sortedArray)
```

```
list: arr
(5 items) [1, 2, 5, 5, 9]
```

input and output:

```
Enter the number of elements: 5
Enter the array elements:
5 2 9 1 5
Sorted Array: [1, 2, 5, 5, 9]
```

- Time Complexity : $O(n^2)$
- Space Complexity : $O(1)$

Result

The program successfully sorts the given list in ascending order while correctly handling empty lists, single-element lists, identical elements, and negative numbers.

3. Write code to modify bubble sort function to stop early if the list becomes sorted before all passes are completed.

Aim

To write a Python program to modify the **Bubble Sort** algorithm so that it **stops early if the list becomes sorted before all passes are completed**.

Algorithm

1. Start.
2. Read the array of elements.
3. Find the number of elements n.
4. Repeat for n-1 passes:
 - Set a flag swapped = False.
 - Compare adjacent elements.
 - If they are in the wrong order, swap them and set swapped = True.
 - After each pass, if swapped = False, stop because the array is already sorted.
5. Display the sorted array.
6. Stop.

code:

```
# Optimized Bubble Sort
def bubbleSort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap elements
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # Stop early if no swaps occurred
        if not swapped:
            break
    return arr

# User input
n = int(input("Enter the number of elements: "))
if n == 0:
    arr = []
else:
    print("Enter the array elements:")
    arr = list(map(int, input().split()))
# Check if correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    sortedArray = bubbleSort(arr)
    print("Sorted Array:", sortedArray)
```

input and output:

```
Enter the number of elements: 5
Enter the array elements:
1 2 2 3 5
Sorted Array: [1, 2, 2, 3, 5]
```

- Time Complexity: **Best: $O(n)$, Average/Worst: $O(n^2)$**
- Space Complexity: **$O(1)$**

Result

The program successfully sorts the array using the **optimized Bubble Sort** algorithm. It stops early when the list is already sorted, reducing unnecessary comparisons.

4. Test Cases:

- Test your optimized function with the following lists:

1. Input: [64, 25, 12, 22, 11]

- Expected Output: [11, 12, 22, 25, 64]

2. Input: [29, 10, 14, 37, 13]

- Expected Output: [10, 13, 14, 29, 37]

3. Input: [3, 5, 2, 1, 4]

- Expected Output: [1, 2, 3, 4, 5]

4. Input: [1, 2, 3, 4, 5] (Already sorted list)

- Expected Output: [1, 2, 3, 4, 5]

5. Input: [5, 4, 3, 2, 1] (Reverse sorted list)

- Expected Output: [1, 2, 3, 4, 5]

Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome.

Examples:

1. Array with Duplicates:

- Input: [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]

- Output: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

2. All Identical Elements:

- Input: [5, 5, 5, 5, 5]

- Output: [5, 5, 5, 5, 5]

3. Mixed Duplicates:

- Input: [2, 3, 1, 3, 2, 1, 1, 3]

- Output: [1, 1, 1, 2, 2, 3, 3, 3]

Aim

To write a Python program to sort an array using the Insertion Sort algorithm while correctly handling duplicate elements and preserving their relative order (stable sorting).

Algorithm

1. Start.
2. Read the array of elements.
3. Traverse the array from the second element to the last.
4. Store the current element as key.
5. Compare key with the previous elements.
6. Shift all elements greater than key one position to the right.
7. Insert key into its correct position.
8. Repeat until the entire array is sorted.
9. Display the sorted array.
10. Stop.

code:

```
# Function to perform Insertion Sort
def insertionSort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        # Move elements greater than key one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

# User input
n = int(input("Enter the number of elements: "))
if n == 0:
    arr = []
else:
    print("Enter the array elements:")
    arr = list(map(int, input().split()))
# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    sortedArray = insertionSort(arr)
    print("Sorted Array:", sortedArray)
```


input and output:

```
Enter the number of elements: 5
Enter the array elements:
1 5 3 4 1
Sorted Array: [1, 1, 3, 4, 5]
```

- Time Complexity : **Best: $O(n)$, Average/Worst: $O(n^2)$**
- Space Complexity : **$O(1)$**

Result

The program successfully counts the number of pairs (i, j) where the elements are equal and the product of their indices is divisible by k, then displays the correct count.

5. Given an array `arr` of positive integers sorted in a strictly increasing order, and an integer `k`. return the `k`th positive integer that is missing from this array.

Example 1:

Input: `arr = [2,3,4,7,11]`, `k = 5`

Output: 9

Explanation: The missing positive integers are [1,5,6,8,9,10,12,13,...].

The 5th missing positive integer is 9.

Example 2:

Input: `arr = [1,2,3,4]`, `k = 2`

Output: 6

Explanation: The missing positive integers are [5,6,7,...]. The 2nd missing positive integer is 6.

Aim

To write a Python program to find the **kth missing positive integer** from a sorted array of distinct positive integers.

Algorithm

1. Start.
2. Read the sorted array `arr` and the integer `k`.
3. Initialize `current = 1` and index `i = 0`.
4. Repeat until the `k`th missing number is found:
 - If `i` is within the array and `arr[i]` equals `current`, increment both `current` and `i`.
 - Otherwise, the current number is missing:
 - Decrement `k` by 1.
 - If `k` becomes 0, display `current` and stop.
 - Increment `current`.
5. Stop.

code:

```
# Function to find the kth missing positive number
def findKthPositive(arr, k):
    current = 1
    i = 0
    while k > 0:
        if i < len(arr) and arr[i] == current:
            i += 1
        else:
            k -= 1
            if k == 0:
                return current
            current += 1
    current += 1

# User input
n = int(input("Enter the number of elements: "))
print("Enter the sorted array elements:")
arr = list(map(int, input().split()))
k = int(input("Enter the value of k: "))
# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    result = findKthPositive(arr, k)
    print("Output:", result)
```

input and output:

```
Enter the number of elements: 5
Enter the sorted array elements:
2 3 4 7 11
Enter the value of k: 5
Output: 9
```

- Time Complexity : $O(n + k)$
- Space Complexity : $O(1)$

Result

The program successfully finds and displays the kth missing positive integer from the given sorted array.

6. A peak element is an element that is strictly greater than its neighbors. Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks. You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array. You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 2

Explanation: 3 is a peak element and your function should return the index number 2.

Example 2:

Input: `nums = [1,2,1,3,5,6,4]`

Output: 5

Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

Aim

To write a Python program to find the **index of a peak element** in a given array using **Binary Search** with **$O(\log n)$** time complexity.

Algorithm

1. Start.
2. Read the array `nums`.
3. Initialize `low = 0` and `high = n - 1`.
4. Repeat while `low < high`:
 - Find the middle index `mid`.
 - If `nums[mid] < nums[mid + 1]`, move to the right half (`low = mid + 1`).
 - Otherwise, move to the left half (`high = mid`).
5. When `low == high`, it points to a peak element.
6. Display the index `low`.
7. Stop.

code:

```
# Function to find a peak element
def findPeakElement(nums):
    left = 0
    right = len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < nums[mid + 1]:
            left = mid + 1
        else:
            right = mid
    return left

# User input
n = int(input("Enter the number of elements: "))
print("Enter the array elements:")
nums = list(map(int, input().split()))
# Check if the correct number of elements is entered
if len(nums) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    index = findPeakElement(nums)
    print("Peak Element Index:", index)
    print("Peak Element:", nums[index])
```

input and output:

```
Enter the number of elements: 4
Enter the array elements:
1 2 3 1
Peak Element Index: 2
Peak Element: 3
```

- Time Complexity : $O(\log n)$
- Space Complexity : $O(1)$

Result

The program successfully finds the index of a peak element in the given array using the Binary Search algorithm.

7. Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or `-1` if `needle` is not part of `haystack`.

Example 1:

Input: `haystack = "sadbutsad"`, `needle = "sad"`

Output: 0

Explanation: "sad" occurs at index 0 and 6.

The first occurrence is at index 0, so we return 0.

Example 2:

Input: `haystack = "leetcode"`, `needle = "leeto"`

Output: -1

Explanation: "leeto" did not occur in "leetcode", so we return -1.

Aim

To write a Python program to find the **index of the first occurrence** of the string **needle** in the string **haystack**. If the substring is not found, return **-1**.

Algorithm

1. Start.
2. Read the strings `haystack` and `needle`.
3. Traverse the `haystack` from index 0 to `length(haystack) - length(needle)`.
4. Compare the substring of `haystack` with `needle`.
5. If they match, display the current index and stop.
6. If no match is found after checking all positions, display -1.
7. Stop.

code:

```
# Function to find the first occurrence of needle in haystack
def strStr(haystack, needle):
    n = len(haystack)
    m = len(needle)

    # Check every possible starting position
    for i in range(n - m + 1):
        if haystack[i:i + m] == needle:
            return i

    return -1

# User input
haystack = input("Enter the haystack string: ")
needle = input("Enter the needle string: ")

# Find the first occurrence
result = strStr(haystack, needle)

print("Output:", result)
```

input and output:

```
Enter the haystack string: leetcode
Enter the needle string: leeto
Output: -1
```

- Time Complexity : $O((n - m + 1) \times m)$
- Space Complexity : $O(1)$

Result

The program successfully finds the index of the first occurrence of the substring needle in haystack. If the substring is not found, it returns -1.

8. Given an array of string words, return all strings in words that is a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string

Example 1:

Input: words = ["mass","as","hero","superhero"]

Output: ["as","hero"]

Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".

["hero","as"] is also a valid answer.

Example 2:

Input: words = ["leetcode","et","code"]

Output: ["et","code"]

Explanation: "et", "code" are substring of "leetcode".

Example 3:

Input: words = ["blue","green","bu"]

Output: []

Explanation: No string of words is substring of another string.

Aim

To write a Python program to find **all strings that are substrings of another string** in the given array of words.

Algorithm

1. Start.
2. Read the array of strings words.
3. Create an empty list result.
4. Traverse each string words[i].
5. Compare it with every other string words[j] where $i \neq j$.
6. If words[i] is a substring of words[j], add it to result and stop checking that word.
7. After checking all words, display the result list.
8. Stop.

code:

```
# Function to find all substring words
def stringMatching(words):
    result = []

    for i in range(len(words)):
        for j in range(len(words)):
            if i != j and words[i] in words[j]:
                result.append(words[i])
                break

    return result

# User input
n = int(input("Enter the number of words: "))

words = []
print("Enter the words:")
for i in range(n):
    words.append(input())

# Find substring words
result = stringMatching(words)

print("Output:", result)
```

input and output:

```
Enter the number of words: 3
Enter the words:
leetcode
et
code
Output: ['et', 'code']
```

- Time Complexity : $O(n^2 \times m)$
- Space Complexity : $O(k)$

Result

The program successfully finds and returns all strings that are substrings of another string in the given array.

9. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

Input:

A list or array of points represented by coordinates (x, y).

Points: [(1, 2), (4, 5), (7, 8), (3, 1)]

Output:

The two points with the minimum distance between them.

The minimum distance itself.

Closest pair: (1, 2) - (3, 1) Minimum distance: 1.4142135623730951

Aim

To write a Python program to find the **closest pair of points** in a set of 2D points using the **Brute Force** approach.

Algorithm

1. Start.
2. Read the list of 2D points.
3. Initialize the minimum distance as infinity.
4. Compare every pair of points using nested loops.
5. Calculate the Euclidean distance between each pair.
6. If the calculated distance is smaller than the current minimum distance:
 - Update the minimum distance.
 - Store the corresponding pair of points.
7. After checking all pairs, display the closest pair and the minimum distance.
8. Stop.

code:

```
import math
# Function to find the closest pair of points
def closestPair(points):
    n = len(points)
    minDistance = float('inf')
    point1 = None
    point2 = None
    # Compare every pair of points
    for i in range(n):
        for j in range(i + 1, n):
            x1, y1 = points[i]
            x2, y2 = points[j]
            distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
            if distance < minDistance:
                minDistance = distance
                point1 = points[i]
                point2 = points[j]
    return point1, point2, minDistance
# User input
n = int(input("Enter the number of points: "))
points = []
print("Enter the coordinates (x y):")
for i in range(n):
    x, y = map(int, input().split())
    points.append((x, y))
# Find closest pair
p1, p2, distance = closestPair(points)
print("Closest Pair:", p1, "-", p2)
print("Minimum Distance:", distance)
```

input and output:

```
Enter the number of points: 4
Enter the coordinates (x y):
1 2
4 5
7 8
3 1
Closest Pair: (1, 2) - (3, 1)
Minimum Distance: 2.23606797749979
```

- Time Complexity : $O(n^2)$
- Space Complexity : $O(1)$

Result

The program successfully finds the closest pair of points and computes the minimum Euclidean distance between them using the Brute Force approach.

10. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7 (6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the sameline?

Given points: P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6

(12.5, 7), P7 (6, 6.5), P8 (7.5, 4.5).

output: P3, P4, P6, P5, P7, P1

Aim

To write a Python program to find the **closest pair of points** using the **Brute Force** approach and to determine the **Convex Hull** of a given set of points using the **Brute Force** method.

Algorithm (Closest Pair of Points – Brute Force)

1. Start.
2. Read the list of points.
3. Initialize the minimum distance as infinity.
4. Compare every pair of points.
5. Compute the Euclidean distance between each pair.
6. If the distance is smaller than the current minimum distance:
 - Update the minimum distance.
 - Store the closest pair.
7. Display the closest pair and minimum distance.
8. Stop.

Algorithm (Convex Hull – Brute Force)

1. Start.
2. Read the set of points.
3. For every pair of points (P_i , P_j):
 - Form a line joining them.
 - Check the position of all other points relative to this line.
4. If all remaining points lie on the same side of the line (or on the line), then the line segment belongs to the convex hull.
5. Repeat for all pairs of points.
6. Arrange the hull points in order.
7. Display the convex hull.
8. Stop.

code:

```
# Function to determine the side of a point
def findSide(p1, p2, p):
    value = (p[1] - p1[1]) * (p2[0] - p1[0]) - (p2[1] - p1[1]) * (p[0] - p1[0])
    if value > 0:
        return 1
    elif value < 0:
        return -1
    else:
        return 0

# Brute Force Convex Hull
def convexHull(points):
    n = len(points)
    if n < 3:
        return []
    hull = set()
    for i in range(n):
        for j in range(i + 1, n):
            left = 0
            right = 0
            for k in range(n):
                if k == i or k == j:
                    continue
                side = findSide(points[i], points[j], points[k])
                if side > 0:
                    left += 1
                elif side < 0:
                    right += 1
            if left == 0 or right == 0:
                hull.add(points[i])
                hull.add(points[j])
    return list(hull)
```

```
# User input
n = int(input("Enter the number of points: "))
points = []
print("Enter the coordinates (x y):")
for i in range(n):
    x, y = map(float, input().split())
    points.append((x, y))
result = convexHull(points)
print("Convex Hull Points:")
for point in result:
    print(point)
```

input and output:

```
Enter the number of points: 8
Enter the coordinates (x y):
10 0
11 5
5 3
9 3.5
15 3
12.5 7
6 6.5
7.5 4.5
Convex Hull Points:
(6.0, 6.5)
(10.0, 0.0)
(15.0, 3.0)
(5.0, 3.0)
(12.5, 7.0)
```

Closest Pair

- Time Complexity : $O(n^2)$
- Space Complexity : $O(1)$

Convex Hull (Brute Force)

- Time Complexity : $O(n^3)$
- Space Complexity : $O(1)$

Result

The program successfully:

- Finds the **closest pair of points** using the Brute Force approach.
- Calculates the **Euclidean distance** between points.
- Determines the **Convex Hull** of the given set of points.
- Handles multiple collinear points by keeping only the extreme points on the hull.

11. Write a program that finds the convex hull of a set of 2D points using the brute force approach.

Input:

A list or array of points represented by coordinates (x, y).

Points: [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]

Output:

The list of points that form the convex hull in counter-clockwise order.

Convex Hull: [(0, 0), (1, 1), (8, 1), (4, 6)]

Aim

To write a Python program to find the **Convex Hull** of a set of 2D points using the **Brute Force** approach.

Algorithm

1. Start.
2. Read the list of 2D points.
3. Initialize an empty list hull.
4. For every pair of points (P_i , P_j):
 - Form a line joining P_i and P_j .
 - Check the position of every other point with respect to this line.
 - If all other points lie on the same side of the line (or on the line), then the edge belongs to the convex hull.
5. Add the valid points to the hull.
6. Remove duplicate points.
7. Arrange the hull points in **counter-clockwise order**.
8. Display the convex hull.
9. Stop.

code:

```
# Function to determine the side of a point
def findSide(p1, p2, p):
    value = (p[1] - p1[1]) * (p2[0] - p1[0]) - (p2[1] - p1[1]) * (p[0] - p1[0])
    if value > 0:
        return 1
    elif value < 0:
        return -1
    else:
        return 0
# Function to find the Convex Hull
def convexHull(points):
    n = len(points)
    if n < 3:
        return []
    hull = set()
    for i in range(n):
        for j in range(i + 1, n):
            left = 0
            right = 0
            for k in range(n):
                if k == i or k == j:
                    continue
                side = findSide(points[i], points[j], points[k])
                if side > 0:
                    left += 1
                elif side < 0:
                    right += 1
            if left == 0 or right == 0:
                hull.add(points[i])
                hull.add(points[j])
```

```
# Sort the hull points
return sorted(hull)
# User input
n = int(input("Enter the number of points: "))
points = []
print("Enter the coordinates (x y):")
for i in range(n):
    x, y = map(int, input().split())
    points.append((x, y))
result = convexHull(points)
print("Convex Hull:", result)
```

input and output:

```
Enter the number of points: 5
Enter the coordinates (x y):
1 1
4 6
8 1
0 0
3 3
Convex Hull: [(0, 0), (4, 6), (8, 1)]
```

- Time Complexity : $O(n^3)$
- Space Complexity : $O(1)$

Result

The program successfully finds the Convex Hull of the given set of 2D points using the Brute Force approach and displays the hull points in counter-clockwise order.

12. You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the TSP. The program should:

1. Define a function `distance(city1, city2)` to calculate the distance between two cities (e.g., Euclidean distance).
2. Implement a function `tsp(cities)` that takes a list of cities as input and performs the following:
 - Generate all possible permutations of the cities (excluding the starting city) using `itertools.permutations`.
 - For each permutation (representing a potential route):
 - Calculate the total distance traveled by iterating through the path and summing the distances between consecutive cities.
 - Keep track of the shortest distance encountered and the corresponding path.
 - Return the minimum distance and the shortest path (including the starting city at the beginning and end).
3. Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Test Cases:

Simple Case: Four cities with basic coordinates (e.g., [(1, 2), (4, 5), (7, 1), (3, 6)])

More Complex Case: Five cities with more intricate coordinates

(e.g., [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)])

Output:

Test Case 1:

Shortest Distance: 7.0710678118654755

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 14.142135623730951

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]

Aim

To write a Python program to solve the **Travelling Salesman Problem (TSP)** using the **Exhaustive (Brute Force) Search** approach by finding the shortest possible route that visits each city exactly once and returns to the starting city.

Algorithm

1. Start.
2. Read the list of city coordinates.
3. Select the first city as the starting city.
4. Generate all possible permutations of the remaining cities.
5. For each permutation:
 - Form a complete path by adding the starting city at the beginning and end.
 - Calculate the total distance by summing the Euclidean distances between consecutive cities.
6. Compare the total distance with the current minimum distance.
7. If a shorter path is found, update the minimum distance and shortest path.
8. After checking all possible routes, display the shortest path and minimum distance.
9. Stop.

code:

input and output:

```
import math
import itertools
# Function to calculate Euclidean distance
def distance(city1, city2):
    return math.sqrt((city1[0] - city2[0])**2 + (city1[1] - city2[1])**2)
# Function to solve TSP using Brute Force
def tsp(cities):
    start = cities[0]
    min_distance = float('inf')
    best_path = []
    # Generate all possible routes
    for perm in itertools.permutations(cities[1:]):
        path = [start] + list(perm) + [start]
        total_distance = 0
        for i in range(len(path) - 1):
            total_distance += distance(path[i], path[i + 1])
        if total_distance < min_distance:
            min_distance = total_distance
            best_path = path
    return min_distance, best_path
# User input
n = int(input("Enter the number of cities: "))
cities = []
print("Enter the coordinates of each city (x y):")
for i in range(n):
    x, y = map(float, input().split())
    cities.append((x, y))
min_distance, best_path = tsp(cities)
print("Shortest Distance:", min_distance)
print("Shortest Path:")
for city in best_path:
    print(city)
```

```
Enter the number of cities: 4
Enter the coordinates of each city (x y):
1 2
4 5
7 1
3 6
Shortest Distance: 16.969112047670894
Shortest Path:
(1.0, 2.0)
(7.0, 1.0)
(4.0, 5.0)
(3.0, 6.0)
(1.0, 2.0)
```

- Time Complexity : $O(n!)$
- Space Complexity : $O(n)$

Result

The program successfully solves the Travelling Salesman Problem (TSP) using the Exhaustive (Brute Force) Search approach by evaluating all possible routes and selecting the one with the minimum total distance.

13. You are given a cost matrix where each element $\text{cost}[i][j]$ represents the cost of assigning worker i to task j . Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function `total_cost(assignment, cost_matrix)` that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function `assignment problem(cost matrix)` that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).

Test Cases:

Input

Simple Case: Cost Matrix:

`[[3, 10, 7],`

`[8, 5, 12],`

`[4, 6, 9]]`

More Complex Case: Cost Matrix:

`[[15, 9, 4],`

`[8, 7, 18],`

`[6, 12, 11]]`

Output:

Test Case 1:

Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker3, task 3)]

Total Cost: 19

Test Case 2:

Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker3, task 2)]

Total Cost: 24

Aim

To write a Python program to solve the **Assignment Problem** using the **Exhaustive (Brute Force) Search** approach by finding the assignment of workers to tasks with the **minimum total cost**.

Algorithm

1. Start.
2. Read the cost matrix.
3. Generate all possible permutations of task assignments.
4. For each permutation:
 - Assign each worker to the corresponding task.
 - Calculate the total assignment cost.
5. Compare the total cost with the current minimum cost.
6. If a smaller cost is found:
 - Update the minimum cost.
 - Store the corresponding assignment.
7. After checking all permutations, display the optimal assignment and the minimum total cost.
8. Stop.

code:

```
# Function to calculate the total cost of an assignment
def total_cost(assignment, cost_matrix):
    cost = 0
    for worker in range(len(assignment)):
        task = assignment[worker]
        cost += cost_matrix[worker][task]
    return cost

# Function to solve the Assignment Problem
def assignment_problem(cost_matrix):
    n = len(cost_matrix)
    min_cost = float('inf')
    best_assignment = None
    # Generate all possible task assignments
    for assignment in itertools.permutations(range(n)):
        cost = total_cost(assignment, cost_matrix)
        if cost < min_cost:
            min_cost = cost
            best_assignment = assignment
    return best_assignment, min_cost
```

```
# User input
n = int(input("Enter the number of workers/tasks: "))
cost_matrix = []
print("Enter the cost matrix:")
for i in range(n):
    row = list(map(int, input().split()))
    cost_matrix.append(row)

# Solve the Assignment Problem
assignment, min_cost = assignment_problem(cost_matrix)
print("Optimal Assignment:")
for worker in range(n):
    print("Worker", worker + 1, "-> Task", assignment[worker] + 1)
print("Total Cost:", min_cost)
```

input and output:

```
Enter the number of workers/tasks: 3
Enter the cost matrix:
3 10 7
8 5 12
4 6 9
Optimal Assignment:
Worker 1 -> Task 3
Worker 2 -> Task 2
Worker 3 -> Task 1
Total Cost: 16
```

- Time Complexity : $O(n! \times n)$
- Space Complexity : $O(n)$

Result : The program successfully finds the optimal assignment of workers to tasks with the minimum total cost using the Exhaustive (Brute Force) Search approach.

14. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

Define a function `total_value(items, values)` that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.

Define a function `is_feasible(items, weights, capacity)` that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.

Test Cases:

Items: 3 (represented by indices 0, 1, 2)

Weights: [2, 3, 1]

Values: [4, 5, 3]

Capacity: 4

Output:

Test Case 1:

Optimal Selection: [0, 2] (Items with indices 0 and 2)

Total Value: 7

Aim

To write a Python program to solve the 0-1 Knapsack Problem using the Exhaustive (Brute Force) Search approach by finding the combination of items that gives the maximum total value without exceeding the knapsack capacity.

Algorithm

1. Start.
2. Read the weights, values, and knapsack capacity.
3. Generate all possible subsets of the given items.
4. For each subset:
 - Calculate the total weight.
 - If the total weight is less than or equal to the capacity:
 - Calculate the total value.
 - If the total value is greater than the current maximum value, update the best selection.
5. After checking all subsets, display the optimal selection and maximum value.
6. Stop.

code:

```
import itertools
# Function to calculate the total value
def total_value(items, values):
    value = 0
    for item in items:
        value += values[item]
    return value
# Function to check if the selected items are feasible
def is_feasible(items, weights, capacity):
    total_weight = 0
    for item in items:
        total_weight += weights[item]
    return total_weight <= capacity
# Function to solve 0-1 Knapsack using Brute Force
def knapsack(weights, values, capacity):
    n = len(weights)
    best_items = []
    max_value = 0
    # Generate all possible subsets of items
    for r in range(n + 1):
        for items in itertools.combinations(range(n), r):
```

```
            if is_feasible(items, weights, capacity):
                value = total_value(items, values)
                if value > max_value:
                    max_value = value
                    best_items = list(items)
    return best_items, max_value
# User input
n = int(input("Enter the number of items: "))
print("Enter the weights:")
weights = list(map(int, input().split()))
print("Enter the values:")
values = list(map(int, input().split()))
capacity = int(input("Enter the knapsack capacity: "))
# Solve the Knapsack Problem
selected_items, max_value = knapsack(weights, values, capacity)
print("Optimal Selection:", selected_items)
print("Total Value:", max_value]
```

input and output:

```
Enter the number of items: 3
Enter the weights:
2 3 1
Enter the values:
4 5 3
Enter the knapsack capacity: 4
Optimal Selection: [1, 2]
Total Value: 8
```

- Time Complexity : $O(2^n \times n)$
- Space Complexity : $O(n)$

Result : The program successfully finds the optimal set of items that maximizes the total value without exceeding the given knapsack capacity using the Exhaustive (Brute Force) Search approach.

